BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

MONITORING OF BITTORRENT TRAFFIC IN LAN

PROJECT REPORT

AUTHOR **SAMUEL OLEKŠÁK**

BRNO 2023

Contents

1 Introduction

This project gives a short overview of BitTorrent network; proposes and evaluates methods for detection and monitoring of BitTorrent traffic on the network; and provides an application for detecting bootstrap nodes, peers and file transfers in captured network traffic.

2 BitTorrent architecture and communication

BitTorrent is a peer-to-peer (P2P) protocol for decentralized file sharing over the Internet.

Shared file is distributed in small chunks called *pieces*. Size of the pieces is not constant and can vary. Common chunk size is 256 KB [3]. When a user wants to download a file using BitTorrent, they first need to obtain a *torrent file* that contains information about the file they want to download, including the file name, size, and a list of tracker servers that help with the distribution of the file and the number of pieces that consist the file. The user can open the torrent file in a BitTorrent client, which connects them to the network and allows them to download and upload pieces of the file from and to other peers on the network.

2.1 Searching for file in the network

BitTorrent network uses two main approaches for searching for files on the network. In case trackers for desired files are known from the torrent file, they can be queried for the location of the file in the network.

In case there is no known tracker for a file, BT-DHT protocol [2] is used to query the DHT. DHT (Distributed Hash Table) is a method to store information in the network in a decentralized manner. Instead of relying on a central server to manage the network, each node in the network stores a portion of the data and can communicate with other nodes to retrieve and share information.

2.2 Bencoding

Bencoding [3] is an encoding format used by BitTorrent protocol. It supports encoding of integers, strings, lists and dictionaries.

As an example, the bencoded string

```
d6:lengthi16e4:name7:PDS.txt12:piece lengthi16384ee
```

would be decoded into the following dictionary:

```
 {
   length:  16,
   name:  PDS.txt,
   piece length:  16384
}
```

2.3 File transferring

The BitTorrent protocol [3] is used for communication between peers, providing a set of rules for requesting and sharing pieces of the file and managing connections with other peers.

In addition, the Micro Transport Protocol (µTP) is used to manage communication between peers in the BitTorrent network. It is a variant of UDP that provides the reliability while minimizing the impact of congestion control on the network. A diagram of the µTP protocol header is shown in fig. 1.

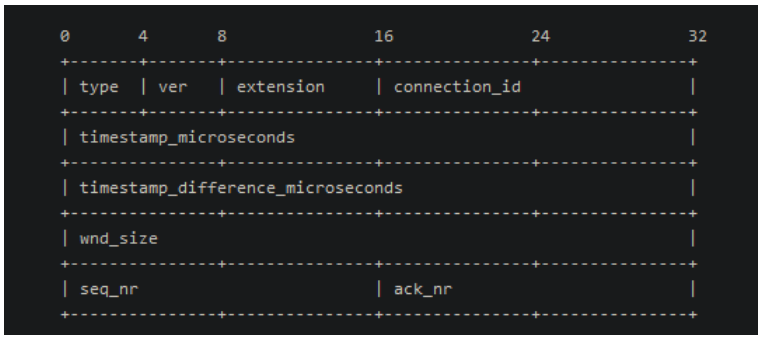

Figure 1: µTP protocol header schema. Taken from [6].

3 Exploring BitTorrent clients

For the experiments, BitTorrent client *qBitTorrent*[1] was used. Each experiment result was captured and saved into packet capture file:

- `q-installation.pcapng` – Captured packets from the first ever startup of the client after installation.
- `q-startup.pcapng` – Captured packets from the subsequent startup of the client.
- `q-download-tcp.pcapng` – Client downloading a file using TCP.
- `q-download-udp.pcapng` – Client downloading a file using µTP.

Capture files were tidied up by applying the display filter `not arp and not nat-pmp`.

4 Methods for BitTorrent traffic detection

4.1 Bootstrap node detection

Using a single method for bootstrap node detection has proven to be unreliable across different clients, therefore multiple detection techniques with their own pros and cons are used to patch each other weaknesses. Bootstrap nodes are extracted from packets containing BT-DTH protocol [2], which is detected by trying to apply bdecoding on every UDP packet. If bdecoding is successful, it is assumed that packet contains BT-DTH protocol[2].

[1] https://www.qbittorrent.org/

[2] It is certainly possible that a non-standard or non-BT-DTH UDP packet containing bencoded payload is encountered so robust error checking has to be established for non-conformant bencoded payloads.

`bs` attribute detection

qBitTorrent client makes detecting bootstrap straight-forward by including `bs:1` in the request arguments dictionary in `get_peers` requests. This behaviour was not observed in *BitTorrent Classic* client.

DNS IP address matching

Bootstrap nodes are often[3] hardcoded in source codes of torrent clients in the form of domain names. IP addresses received from translation of these domain names through DNS communication can be saved. If a DHT-BT communication takes place between client's IP address and IP address received from DNS, there is high likelihood that node is a bootstrap node. Con of this approach is that some clients could cache DNS responses, which would render this method inconsistent, although this behaviour was observed by neither of the BitTorrent clients.

Some nodes could theoretically be falsely labelled bootstrap if they are received in the form of a domain name instead of IP address, which does not occur in standard BT-DHT.

Well known domains and keyword detection

A list of common trackers can be obtained on the Internet[4] or manually constructed. If a client queried one of these well-known trackers, it is highly possible, albeit not certain, that a user is using a BitTorrent client and connecting to the network. Downside of this approach is that the list needs constant upkeep to be up-to-date at all times, unless automated methods, like web scraping, are used.

When observing some of the common tracker domain names, it is apparent that they often contain keywords including *dht*, *torrent* and *tracker*. These keywords can be used to detect domain names belonging to trackers. However, this approach has a high false negative rate.

Tracking received nodes

An approach where, each node received by the BT-DHT protocol would be saved into an array. Every BT-DHT packet sent from the client would be inspected and receiver node would be searched in the array of received nodes. If a BT-DHT message was sent to a node that is not saved in the received node array, we can assume, it was obtained by a different mean and therefore is probably a bootstrap node.

4.2 Peer detection

In the context of this section, we understand the term peer as a node in the BitTorrent network. BT-DHT protocol command `get_peers` is used to find other peers in the network. By inspecting every BT-DHT packet, we can extract the list of peers along with their ID, IP address and port. This information is available in the `nodes` section in the `get_peers` command responses. Number of connections is counted as the number of unique transaction

[3] See source code of *Deluge* client (https://dev.deluge-torrent.org/browser/deluge/core/preferencesmanager.py?rev=415979e2f76658c4e325b7854f0a66206f0bc5c8#L320) or *qBitTorrent* client (https://github.com/qbittorrent/qBittorrent/blob/7397c80837eddd4987c46dd30dc3e514c1213c58/src/base/bittorrent/sessionimpl.cpp#L1853)

[4] For example https://github.com/ngosang/trackerslist

IDs observed in the BT-DHT packets that have at least two packets associated with them (single packet means the other side did not respond, therefore there was no connection).

4.3 File download tracking

Files are transferred in pieces using the BitTorrent protocol, which can use either TCP or UDP. File transfer is preceded by a BitTorrent handshake (fig. 2).

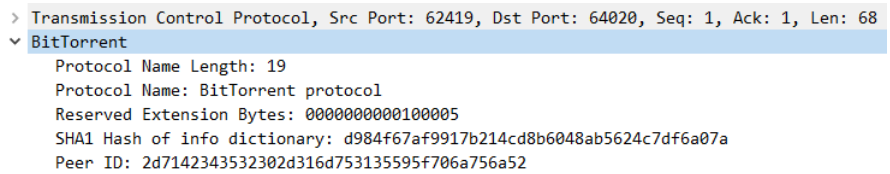

Figure 2: Bisection of BitTorrent handshake packet. The protocol is easily identifiable by the `\x13BitTorrent protocol` field. Handshake contains the `info_hash` of the requested file.

After connection is established using handshake, content infohash is matched with the IP address and port of the handshake counterparty. µTP uses connection id field to distinguish various streams.

Sent file pieces start with BitTorrent protocol header with `piece` command (type 7). Pieces usually span across multiple packets and need to be reconstructed back. Sometimes BitTorrent protocol headers can occur in the middle of payload and need to be taken care of by precisely calculating the number of expected and received bytes in the flow.

4.4 Routing table construction

DHT protocol is to search for files with no tracker available. To aid in the search of files, routing table with information about other nodes in the network is constructed. The routing table consists of so-called buckets. Nodes are grouped together in buckets according to their distance to the client node. Distance between two hashes is given by the number of matching bits of the hashes' prefixes.

Routing table can be constructed from the captured communication by examining the BT-DTH packets' payloads. Client can use multiple ids so a separate routing table is constructed for each of the client used ids. BT-DHT request packets with `get_peers` command contain the id of the current client id. Responses can be matched to these requests by using the transaction id field. Responses contain information about other nodes in the network, including their id, IP address and port.

To distinguish incoming and outgoing `get_peers` requests, the IP address of the client has to be established. IP address of the client is selected as the address that has the most occurrences in destination or source IP address fields. Packet with the most occurrences will certainly be the client, unless they are using multiple IPs to communicate.

Another solution to select the sender would be to select the sender of the first BT-DHT packet. This method has an obvious downside – if the process of capturing was started in the middle of BT-DTH communication, not before it, the client IP might be set incorrectly. Another downside might be if the client was using multiple IPs to communicate with the rest of the network.

```
Routing table of 8a46b2e996172b1abcb4a1808e416a994d1b25a1

distance 0
54c839487a72e4864ffce2743371f5025b81d52d 63639 87.249.132.168
711068aeb5d688be2b8e2cf1472bded6403b50b3 45084 180.252.170.6

distance 1
fecde844b39d1674054da0cd284bd7ec9001fdb1 55279 247.78.214.25
f6f4cb06065b665ac10ee6e31bb5ebd46b6a7361 58490 85.62.176.232

distance 3
9ae78eabfba13dea8826a102fdb9cdfe6bd76b9d 11707 48.165.112.254
9af41daee3aabafd2b03eabe6c68c5b35f72bdb7 1100  5.15.168.61
9443e9e9ebb3a6db3c870c3e99245e0d1c06b7f1 49751 86.153.2.174

distance 8
8ab66b81df9c9115b39b5fc2d859750e56827742 29746 219.186.101.49
```

5 Implemented application

Application was implemented in Python. `Scapy` library is used to parse packet capture files. Application expects pcap files to be supplied using the `-pcap` parameter. No `tshark` preprocessing into CSV format is needed.

Detailed instructions on how to run the application are printed upon providing either `-h` or `-help` parameter or can be found in the included `Readme.txt` file.

From the previous chapter all detection methods except *Well known domains and keyword detection* and *Tracking received nodes* were implemented.

6 Testing and evaluation

Our program was not only evaluated on IPv4 as we were not able to force the *qBitTorrent* client to use IPv6. For testing the downloading detection, two torrent files were used `mnist.torrent` and `images.torrent`.

6.1 Bootstrap nodes testing

Command: `python3 bt-monitor.py -pcap pcap/q-installation.pcapng -init`

Hardcoded bootstrap node domain names[5] were fetched from the source code. They were manually translated to IP addresses using `nslookup`. Five hardcoded domain names translated using `nslookup` tool were translated into 6 IPv4 and 2 IPv6 addresses. Although the client requested `AAAA` records from DNS, he didn't fire any BT-DHT queries at it. All 6 IPv4 addresses were successfully caught by our `bt-monitor` along with their corresponding port which is hardcoded in the source code of the client as well. Three of those six hosts identified by IPv4 addresses responded to the queries therefore the tool also displays ID for them. Remaining three hosts' requests went unanswered.

6.2 Peer testing

Command: `python3 bt-monitor.py -pcap pcap/q-startup.pcapng -peers`

[5] https://github.com/qbittorrent/qBittorrent/blob/7397c80837eddd4987c46dd30dc3e514c1213c58/src/base/bittorrent/sessionimpl.cpp#L1853

Our program returned 75 peers, some of which had Unknown IDs. Entries were validated by inspecting the pcap file with Wireshark. Entries with unknown IDs were produced when contacting bootstrap nodes and not receiving an answer. Their IP address was known from DNS resolution and port was hardcoded in the source code of the client. Entries with known IDs were also validated using Wireshark.

6.3 Download testing

Downloading was tested on two files with different piece size and on either TCP or µTP protocols. Metadata about torrent files were extracted using the tool `torrent_parser`[6].

Download using TCP

Command: `python3 bt-monitor.py -pcap pcap/q-download-tcp.pcapng -download`

`q-download-tcp.pcapng` contains packets captured during the download of `images.torrent`. Content consists of 589 pieces of size 32768 B. Total size of the file is 17 614 527 B. The file was downloaded using TCP. Our tool reported the following information upon processing the packet capture file:

```
Infohash: d984f67af9917b214cd8b6048ab5624c7df6a07a
Size: 18449485 B
Pieces: 585
Contributors:
- 185.149.90.114:51033 225 pieces
- 185.203.56.27:64020 362 pieces
```

The infohash and contributor IP address and port, match up the information obtained from Wireshark. The total size of the content is reported to be a bit larger than the actual size but is within 5 % difference. Number of pieces roughly matches up – our tool only missed 4 out of 589 chunks.

Download using µTP

Command: `python3 bt-monitor.py -pcap pcap/q-download-udp.pcapng -download`

For the µTP test, another file was used – the MNIST dataset from `mnist.torrent`. Archive consists of 60 pieces of length 262 144 B and the total size is 15 683 414 B. The result from our app was following:

```
Infohash: 4d563087fb327739d7ec9ee9a0d32c4cb8b0355e
Size: 15781718 B
Pieces: 60
Contributors:
- 20.29.93.225:25000 32 pieces
- 72.21.17.2:25000 32 pieces
```

Infohash, contributor IP addresses and ports and number of pieces match up exactly. As we can see the tool could handle two separate contributors for the same piece. The actual size is only different by 0,6 % to reported size.

[6] https://github.com/7sDream/torrentq_parser/tree/master

6.4 Routing table testing

Command: `python3 bt-monitor.py -pcap pcap/q-startup.pcapng -rtable`

In the pcap file, client is using two different IDs, one seemingly to communicate with bootstrap nodes and one for everything else. Therefore two routing tables were constructed for each of the client's IDs. By selecting a few nodes and searching for them in Wireshark BT-DHT communication and manually calculating the distance, we found out that reported results correspond to the captured packets.

7 Results

Our `bt-monitor` was able to successfully detect bootstrap nodes, peers and construct routing table from the captured communication of the *qBitTorrent* client. Reported quantitative information about transferred files (total size, pieces count) is within reasonable tolerance and qualitative information (file hashinfo, contributor addresses and port) do match exactly the capture packets. There are obviously a lot of edge cases that can happen skewing the reported information. The portability of this tool across various BitTorrent clients is only partial.

8 Conclusion and contribution

Due to the often non-standard and inconsistent nature of BitTorrent clients, which is likely a consequence of deliberate decentralization, multiple methods need to be interleaved to provide somewhat robust architecture for BitTorrent traffic detection across the vast spectrum of available clients.

The goal of this project was to explore BitTorrent architecture and how various clients communicate with the BitTorrent network during initialization, downloading and seeding of files; propose methods for detecting BitTorrent traffic in network communication; implement and describe an application for performing these methods on packet capture files; and evaluate these methods with regard to reliability across different BitTorrent clients.

9 Acknowledgements

Torrent files used for this project were sourced from *Academic Torrents*[7] [4, 5]. Two torrent files were used, one of them contained the MNIST dataset[8] [1], second one[9] contained two images, which are in public domain.

[7] https://academictorrents.com
[8] https://academictorrents.com/details/4d563087fb327739d7ec9ee9a0d32c4cb8b0355e
[9] https://academictorrents.com/details/d984f67af9917b214cd8b6048ab5624c7df6a07a

Bibliography

[1] AL., A. LeCun et. MNIST. Available at: http://yann.lecun.com/exdb/publis/index.html#lecun-98.

[2] ANDREW LOEWENSTERN, A. N. *DHT protocol* [https://www.bittorrent.org/beps/bep_0005.html]. January 2008. Accessed: April 16, 2023.

[3] COHEN, B. *The BitTorrent Protocol Specification* [http://www.bittorrent.org/beps/bep_0003.html]. January 2008. Accessed: April 16, 2023.

[4] COHEN, J. P. and LO, H. Z. Academic Torrents: A Community-Maintained Distributed Repository. In: *Annual Conference of the Extreme Science and Engineering Discovery Environment*. 2014. DOI: 10.1145/2616498.2616528. Available at: http://doi.acm.org/10.1145/2616498.2616528.

[5] LO, H. Z. and COHEN, J. P. Academic Torrents: Scalable Data Distribution. In: *Neural Information Processing Systems Challenges in Machine Learning (CiML) workshop*. 2016. Available at: http://arxiv.org/abs/1603.04395.

[6] NORBERG, A. *UTorrent transport protocol* [http://www.bittorrent.org/beps/bep_0029.html]. June 2009. Accessed: April 24, 2023.